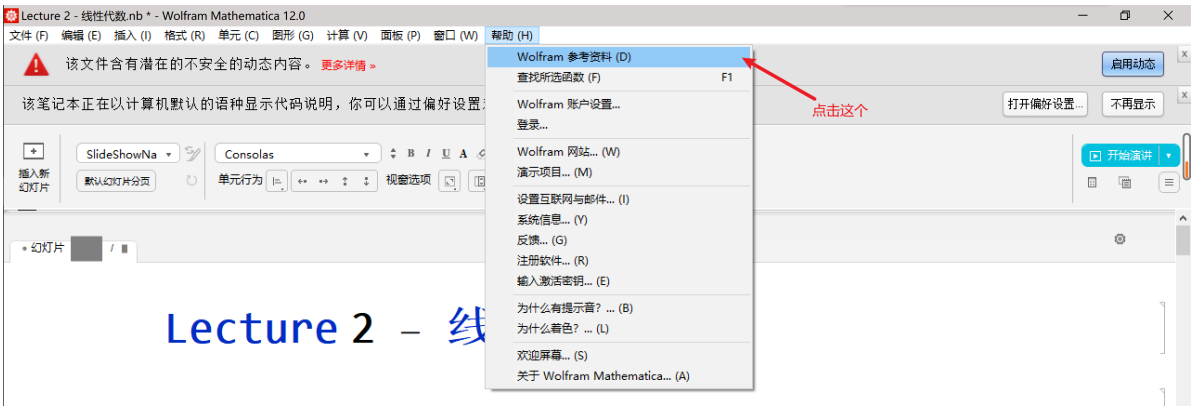
说明：本篇入门笔记以官网文档为主，只是起到辅助作用，需要入门的dalao可以自行学习。


**1. 进入官网文档**
**2. 快速编程入门**
- 2.1 交互式用法
- 2.2 笔记本文档
- 2.3 内置函数
- 2.4 符号表达式
- 2.5 列表
- 2.6 迭代器
- 2.7 赋值
- 2.8 模式
- 2.9 函数定义
- 2.10 纯函数
- 2.11 函数应用
- 2.12 函数式与运算符
- 2.13 选项
- 2.14 图形
- 2.15 交互式界面
- 2.16 过程
- 2.17 数值
- 2.18 字符串
- 2.19 关联
- 2.20 自然语言输入
- 2.21 真实世界实体
- 2.22 解释器
- 2.23 云部署
- 2.24 制作文档
- 2.25 管理会话
- 2.26 外部连接


# 1. 进入官网文档

方法一：

方法二：


Lecture 2 - 线性代数.nb * - Wolfram Mathematica 12.0
文件 (F) 编辑 (E) 插入 (I) 格式 (R) 单元 (C) 图形 (G) 计算 (V) 面板 (P) 窗口 (W) 帮助 (H)
该文件含有潜在的不安全的动态内容。 更多详情 »
启用动态
该笔记本正在以计算机默认的语种显示代码说明，你可以通过偏好设置
打开偏好设置...
不再显示
Wolfram 参考资料 (D)
查找所选函数 (F) F1
Wolfram 账户设置...
登录...
Wolfram 网站... (W)
演示项目... (M)
设置互联网与邮件... (I)
系统信息... (Y)
反馈... (G)
注册软件... (R)
输入激活密钥... (E)
为什么有提示音? ... (B)
为什么着色? ... (L)
欢迎屏幕... (S)
关于 Wolfram Mathematica... (A)
点击这个
SlideShowNa
Consolas
开始演讲
幻灯片
Lecture 2 – 线

Wolfram 语言参考资料中心 - Wolfram Mathematica 12.0
文件 (F) 编辑 (E) 插入 (I) 格式 (R) 单元 (C) 图形 (G) 计算 (V) 面板 (P) 窗口 (W) 帮助 (H)
Wolfram语言与系统 | 参考资料中心
核心语言与结构
数据操作与分析
可视化与图形
机器学习
符号与数值计算
字符串与文本
图与网络
图像
几何学
声音
知识表示与自然语言
与时间有关的计算
地理数据与计算
科学和医学的数据与计算
工程数据与计算
金融数据与计算
社会、文化与语言数据
高等数学计算
文档与演示
用户界面构建
系统操作与设置

工作流程
在笔记本中执行的操作
创建文档和演示文稿
创建界面
使用 Wolfram 语言
处理数据
处理图形、图像和声音
在云端执行的操作
部署到网络和移动设备
存储库、共享和发布
与其他系统连接
设置与管理
特定应用领域
资源
点击进入在线教程
快速编程入门
入门书与课程
存档与知识库
开发人员资源
Wolfram 语言产品
Wolfram U

最终学习界面：

接下来就可以**开始学习**啦！！

> 因为网站在国外，可能会有点慢，等待的时候可以回顾学过的内容，不要浪费时间。

# 2. 快速编程入门

## 2.1 交互式用法

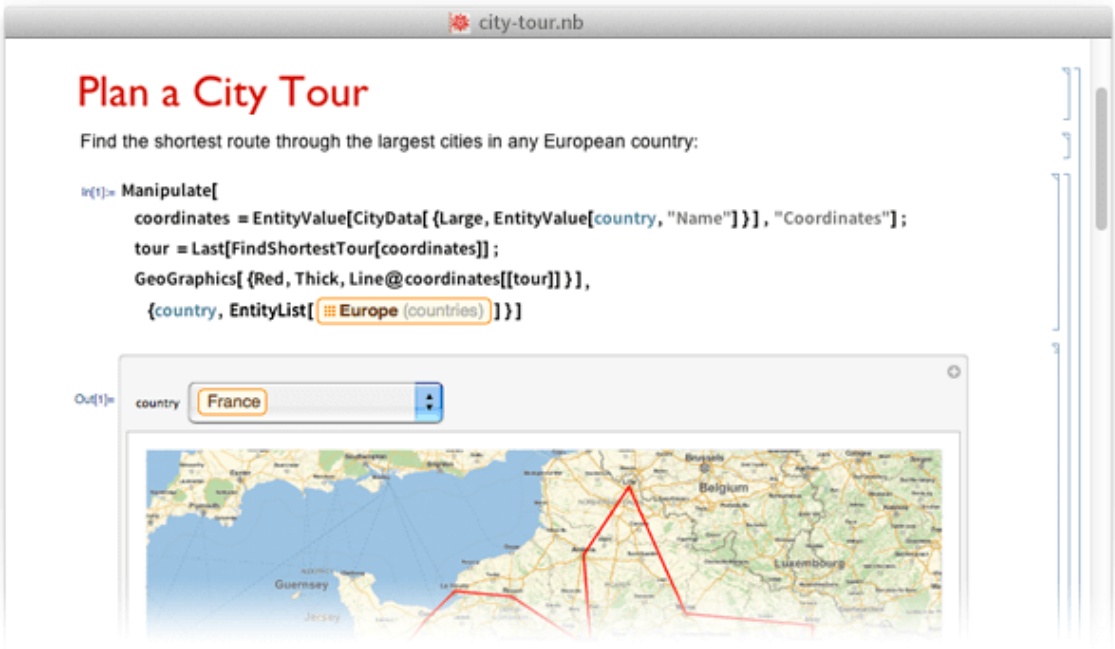
在桌面或网页上的一个 Wolfram 笔记本，只需敲入输入，然后按 `SHIFT`+`ENTER` 来进行计算

```
In[1]:= 2 + 2

Out[1]= 4

In[2]:= Range[10]
        范围

Out[2]= {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

> In[n] 和 Out[n] 标签会连续标记输入和输出. 您可以使用 % 指定最近的输出，但是最好是定义一个变量.

## 2.2 笔记本文档

1. Wolfram 笔记本在电脑桌面或网页上均可使用, 它能将代码与文本、图形和界面等一起混合使用




2. 笔记本是按**单元**组织的，并由右边的**方括号**指定
3. **双击单元方括号**打开或关闭单元组
4. 在单元间点击可以获取**水平插入条**，以便创建一个新的单元
5. 拷贝、粘贴、删除任何单元集合等.选中任何单元集合并按住 SHIFT + ENTER 对输入进行计算.

## 2.3 内置函数

- Wolfram 语言中有近 [6,000个内置函数](). 每个函数名称的首字母均为大写

```
In[5]:= Range[20]
        范围
Out[5]= {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}

In[6]:= NestList[f, x, 5]
        嵌套列表
Out[6]= {x, f[x], f[f[x]], f[f[f[x]]], f[f[f[f[x]]]], f[f[f[f[f[x]]]]]}
```

> *PYTHON* 程序员备注：
> Wolfram 语言延续了一致统一设计且附带了完整整合的内置函数，而无需从不同设计和约定的不同模块和程序 库加载功能.

- 函数参数**始终用逗号分隔.**

  *PYTHON* 程序员备注：
  在 Wolfram 语言中，函数参数**使用方括号 [...] 而不是括号 (...)**括号用于分组.

- 将鼠标悬停在任何内置函数名下，可以显示 简介或 文档和范例

> *PYTHON* 程序员备注：
> Wolfram 语言比 Python 拥有更多且范围广的内置功能，可更简单和快速书写代码，并且 **ExternalEvaluate** 还可用于整合 Python 代码. 你可以通过 Python 用 Wolfram 客户端库中的 **language.wl** 在 Python 中直接访问内置 Wolfram 语言函数和符号.

## 2.4 符号表达式

Wolfram 语言中的一切都是符号表达式.

所有符号表达式都具有相同的基本结构：*head*[*arguments*]

{a, b, c} ····· List[a, b, c]

2+2 ····· Plus[2, 2]

$x^2+3y^3$ ····· Plus[Power[x, 2], Times[3, Power[y, 3]]]

····· Graphics3D[Sphere[]]

函数的参数可以是任何符号表达式：

---

Wolfram 语言是完全符号式，因此 “未定义变量” 总保持原样：

In[1]:= x

Out[1]= x ← 未变化

> *PYTHON* 程序员备注：
> Wolfram 语言的符号表达式让程序员可以在其计算中不间断地将未定义变量作为符号使用. 未赋值的变量在 Python 中被认为是 “undefined”，且试图使用未定义变量会造成产生 **NameError**.

---

FullForm 总是显示底层结构.
Head 总是给出一个表达式；Length 则给出参数数目.

## 2.5 列表

列表 在 Wolfram 语言中用 {...} 来表示，它们可以是任何类型的表达式

> PYTHON 程序员备注：
> 当通过 Python 用 Wolfram 客户端库 编写 Python 时，例如 **list**、**set** 和 **frozenset**，以及生成器函数等分类被序列化为 **List**.

其中可以包含各种类型的表达式：

In[1]:= {3, 4, 5, 7/8, x, y, $x^2+3x^3$, {a, b, c}, }

列表部分的**索引从 1 开始**，可以使用 [[ ... ]] 进行提取：

In[2]:={a, b, c, d}[[3]]

Out[2]=c

---

**负索引**从列表的结尾向前开始计数：

In[3]:={a, b, c, d, e, f}[[-3]]

Out[3]=d

> *PYTHON* 程序员备注：
> Wolfram 语言列表用大括号 **{...}** 表示，然而 Python 列表使用相同方括号 **[...]** 语法同时表示列表和索引. The Wolfram 语言通过用 **[[...]]** 表示索引来避免混淆，并运用了从 1 而不是从 0 开始的直观的索引系统.

---

许多操作直接 “线性” 作用于列表：

In[1]:={1, 2, 3} + 2

Out[1]={3, 4, 5}

In[2]:={a, b, c} + {x, y, z}

Out[2]={a + x, b + y, c + z}

> *PYTHON* 程序员备注：
> 在 Wolfram 语言中，**+** 仅仅用于算数加法. 对于字符串串联等使用 **<>**.

---

列表中的 “跨度” 使用 ;;

In[1]:={a, b, c, d, e, f}[[2 ;; 4]]

Out[1]={b, c, d}

> PYTHON 程序员备注：
> **;;** 跨度在 Wolfram 语言中与 Python 的 **:** 切片类似，但概化为多维数组.

## 2.6 迭代器

创建一个前 10 个平方数的表格：

In[1]:=Table[x^2, {x, 10}] ← *x* 从 1 到 10

Out[1]={1, 4, 9, 16, 25, 36, 49, 64, 81, 100}

Wolfram 语言中的许多函数使用标准的 “迭代器规范”：*min*、*max*、*step*：

In[2]:=Table[f[x], {x, 4, 20, 2}]

Out[2]={f[4], f[6], f[8], f[10], f[12], f[14], f[16], f[18], f[20]}

另一种迭代器规范只给出一个显式值列表：

In[3]:=Table[f[x], {x, {5, 10, 20, 10, 5}}]

Out[3]={f[5], f[10], f[20], f[10], f[5]}

> *PYTHON* 程序员备注：
> 这样使用 **Table** 与在 Python 中使用 **[... for ... in ...]** 列表推导式结构类型，但有更丰富的语义.

---

制作一个嵌套表格：

外索引 ↓ 内索引 ↓

In[1]:=

```
Table[ i / j, { i, 4 }, { j, 2 }]
```

Out[1]=

$$\left\{\left\{1, \frac{1}{2}\right\}, \{2, 1\}, \left\{3, \frac{3}{2}\right\}, \{4, 2\}\right\}$$

## 2.7 赋值

可以使用 = 进行赋值

In[1]:=**x = 7**

Out[1]=**7**

此时为 “立即” 赋值.

也可以**使用延迟赋值，当被调用时，值会被重新计算**:

In[2]:=**t := Now**

In[3]:=**t**

Out[3]= Sat 31 May 2014 23:10:55

In[4]:=**t**

Out[4]= Sat 31 May 2014 23:10:59 ← 重新计算

> *PYTHON* 程序员备注:
> Wolfram 语言中，延迟赋值生成的变量在你每次使用的时候将动态重新计算其值. 为实现相同功能 Python 则需要使用更复杂的 promise 对象.

---

**清除赋值:**

In[1]:=**x =.**

In[1]:=**t =.**

> *PYTHON* 程序员备注:
> 在 Wolfram 语言中的 **=.** 与 Python 的 **del** 操作符有相同作用.

---

用 Module 局部化变量:

In[2]:=**Module[{ a = 1}, a + 8]**

Out[2]=**9**

> *PYTHON* 程序员备注:
> Python 函数创建本地范围变量与 Wolfram 语言中的 **Module** 类似。

---

变量名通常以小写字母为首字母，大写开头的变量一般是特别为内置对象所保留.

**abc ...** ← 用户定义的　　**Abc ...** ← 内置的

> *PYTHON* 程序员备注：
> 在 Python 的命名习惯中，大写名称特指class；在 Wolfram 语言中，大写名称可表示任意类型的内置结构.

## 2.8 模式

> *PYTHON* 程序员备注：
> Wolfram 语言的模式语言允许你对任意符号结构的模式进行描述，能够将强大的 regex-like 操作泛用与任意表达式和任意格式的数据.

模式代表表达式的类型. 基本的模式结构 _（读作 “blank”）代表任意表达式.

查找列表中匹配模式 f[_] 的表达式：

In[1]:=**Cases[{f[1], g[2], f[5], g[3]}, f[_]]**

Out[1]={f[1], f[5]}

---

*x*_（*x*:_ 的缩写）代表一个模式，其值将被命名为 *x*：

“规则”⟶ 可以输入为 ->

In[2]:=**Replace[f[100], f[*x*_] → x+5]**

Out[2]=105

---

/. 表示 “全部替代”：

In[3]:={**f[1], g[2], f[5], g[3]} /. f[*x*_] → x+5**

Out[3]={6, g[2], 10, g[3]}

> *PYTHON* 程序员备注：
> 在此显示的该类结构的元程序，将**代码和数据等同对待**，是 Wolfram 语言符号结构的独有特点.

---

__ （“两个 blank”） 代表任意表达式序列：

In[1]:=**Cases[{f[1, 2], f[1], g[3]}, f[__]]**

Out[1]={f[1, 2], f[1]}

---

*a* | *b* | *c* 代表 *a*、*b* 或 *c*：

In[1]:=**Cases[{f[1], g[2], f[2], f[5], g[3]}, f[1 | 5]]**

Out[1]={f[1], f[5]}

> *PYTHON* 程序员备注：
> | 在 Wolfram 语言模式中的使用与其在 Python 常用表达式中的用法相近. 但是，Wolfram 语言中，| 不仅用于字符串，还可用于任意类型的符号模式匹配.

它也适用于标头：

In[2]:=**Cases[{f[1], g[2], f[2], f[5], g[3]}, (f | g)[2]]**

Out[2]={g[2], f[2]}

---

_*h* 代表任何有标头 *h* 的表达式：

In[1]:=**Cases[{1, 2.5, 3.5, 4}, _Real]**

Out[1]={2.5, 3.5}

---

:> 是一个延迟规则；类似于规则中的 :=

## 2.9 函数定义

在 Wolfram 语言中，函数定义只是给出模式变换规则的赋值.

定义一个带有命名为 *x* 和 *y* 的两个参数的函数：

In[1]:=**f[*x*_, *y*_] := *x* + *y***

> *PYTHON* 程序员备注：
> 在 Wolfram 语言的基本函数定义中，**参数名称后跟随 _ (“blank”)，允许参数与任意符号式模式一致.** 实现了强大且灵活的函数定义，这对于默认的 Python 是不可能的. 你可以通过 Python 用 Wolfram 客户端库中的 **session.function** 指定 Wolfram 语言-函数从而在 Python 中使用。

使用定义：

In[2]:=**f[4, a]**

Out[2]=4 + a

清除定义：

In[3]:=**Clear[f]**

---

函数定义可以按序指定其值：

In[1]:=**f[1] = 2**

Out[1]=2

In[2]:=**f[2] = 3**

Out[2]=3

任何未定义的，还保留原样：

In[3]:={**f[1], f[2], f[3], f[4]**}

Out[3]={2, 3, f[3], f[4]}

您可以使用 /; 限制定义，使其只适用于某种条件：

In[4]:=**f[ x_, y_ ] := x − y /; x > y**

> *PYTHON* 程序员备注：
> 在 Wolfram 语言中可使用与 **if** 语句相同的 **/;** 应用于函数，允许函数自由根据条件自动选择定义.

---

您可以**定义任何左手边的变换"解构"参数的便捷方式**：.

In[1]:=**f[{ x_, y_ }, z_ ] := { x + z, y − z }**

In[2]:=**f[{ 5, 6 }, q ]**

Out[2]={5 + q, 6 − q}

---

Wolfram 语言没有所谓的类型；一切都是符号表达式. 表达式的"标头"可被作为普遍意义上的类型标志符使用.

使用一个"*u* 对象"或"*v* 对象"，给出 *f* 的不同定义：

In[1]:=**f[ u[ x_ ] ] := x + 1**

In[2]:=**f[ v[ x_ ] ] := x − 1**

> *PYTHON* 程序员备注：
> Wolfram 语言函数可使用内置的模式匹配来根据其任意参数的符号结构来更改行为. 在 Python 中则需要相对复杂的自定义代码或第三方程序库.

## 2.10 纯函数

Wolfram 语言允许调用 纯函数，由 & 结尾，第一个参数由 # 指明：
（这些也被称为匿名函数、拉姆达表达式等.）

创建一个加1的纯函数：

In[1]:=

**( # + 1 ) &**

Out[1]=

#1 + 1 &

如果**纯函数以表达式的标头给出**，会对**参数**应用该函数：

In[2]:=

**( # + 1 )& [ 50 ]**

Out[2]=

51

这是一个具有**多个参数的函数**：

In[3]:=

**{ #2, 1 + #1, #1 + #2 }& [ a, b ]**

Out[3]=

```
{b, 1 + a, a + b}
```

可用其它方法指定函数：

In[4]:=

```
Function[{x, y}, {y, 1 + x, x + y}][a, b]
```

Out[4]=

```
{b, 1 + a, a + b}
```

---

很多**内置函数常使用纯函数**：

In[1]:=

```
Select[{1, 4, 6, 8, 10, 15}, # > 7 &]
```

Out[1]=

```
{8, 10, 15}
```

In[2]:=

```
NestList[f[#, #] &, a, 3]
```

Out[2]=

```
{a, f[a, a], f[f[a, a], f[a, a]], f[f[f[a, a], f[a, a]], f[f[a, a], f[a, a]]]}
```

## 2.11 函数应用

将一个函数“映射”到多个表达式是很常见的：

In[1]:=

```
Map[f, {a, b, c, d}]
```

Out[1]=

```
{f[a], f[b], f[c], f[d]}
```

/@（“斜杠 艾特”）是 Map 的简写形式：

In[2]:=

```
f /@ {a, b, c, d}
```

Out[2]=

```
{f[a], f[b], f[c], f[d]}
```

这里使用一个 纯函数：

In[3]:=

```
{#, #} &/@ {a, b, c, d}
```

Out[3]=

```
{{a, a}, {b, b}, {c, c}, {d, d}}
```

---

Apply 将一个函数应用于多个参数:

In[1]:=

```
Apply[f, {a, b, c, d}]
```

Out[1]=

```
f[a, b, c, d]
```

---

表达式所带的"层"对应于需要提取部分的索引号. 像 Map 这样的函数可以在指定的层进行操作.

Map 默认在第一层操作:

In[1]:=

```
Map[f, {{a, b}, {c, d}}]
```

Out[1]=

```
{f[{a, b}], f[{c, d}]}
```

该操作只在第二层进行:

In[2]:=

```
Map[f, {{a, b}, {c, d}}, {2}]
```

Out[2]=

```
{{f[a], f[b]}, {f[c], f[d]}}
```

---

@@ 等价于 Apply，默认在第0层操作:

In[1]:=

```
f @@ {{a, b}, {c, d}}
```

Out[1]=

```
f[{a, b}, {c, d}]
```

@@@ 是指"在第一层应用":

In[2]:=

```
f @@@ {{a, b}, {c, d}}
```

Out[2]=

```
{f[a, b], f[c, d]}
```

@ 是指普通函数应用:

In[3]:=

```
f @ x
```

Out[3]=

```
f[x]
```

以下哪项表达式计算为 **g[{a, b, c}]**?

g@{a, b, c} 正确.

g /@ {a, b, c}

g @@ {a, b, c}

---

以下哪项表达式计算为 **{{f[a], f[b]}, {f[c], f[d]}}**?

f @@ {{a, b}, {c, d}}

f @@@ {{a, b}, {c, d}}

Map[f, {{a, b}, {c, d}}, {2}] 正确.

---

以下哪项计算为 **{{{f[a], f[b]}}, {{f[c], f[d]}}}**?

Map[f, {{{a, b}}, {{c, d}}}]

Map[f, {{{a, b}}, {{c, d}}}, {3}] 正确.

f @@@ {{{a, b}}, {{c, d}}}

## 2.12 函数式与运算符

Wolfram 语言中的许多内置函数可使用“函数式”或“运算符”格式.

以下给出列表中最接近 6.3 的元素:

In[1]:=

```
Nearest[{1, 4, 6, 8, 10, 15}, 6.3]
```

Out[1]=

```
{6}
```

以下设置一个“最接近函数”，可应用于任何指定的值:

In[2]:=

```
Nearest[{1, 4, 6, 8, 10, 15}]
```

Out[2]=

```
NearestFunction[ Data points: 6 Input: scalar ]
```

使用一个指定值，将给出一个特定的结果:

In[3]:=

```
%[6.3] ← % 指的是前一个输出
```

Out[3]=

```
{6}
```

---

若未提供显式数据，符号式则表示选择操作：

In[1]:=

```
Select[# > 7 &]
```

Out[1]=

```
Select[#1 > 7 &]
```

若提供一个显式数据，则会给出结果：

In[2]:=

```
Select[# > 7 &][{1, 4, 6, 8, 10, 15}]
```

Out[2]=

```
{8, 10, 15}
```

## 2.13 选项

> PYTHON 程序员备注：
> Wolfram 语言选项与 Python 中的可选参数类似，除了选项使用 -> 而不是 =，且支持符号值.

许多内置函数都有会影响其操作或输出的“选项”：

In[1]:=

```
Framed[x, Background → LightYellow]
```

Out[1]=

```
x
```

Options 给出一个函数的默认选项：

In[2]:=

```
Options[Framed]
```

Out[2]=

```
{Alignment → Automatic, Background → None, BaselinePosition → Automatic,
 BaseStyle → {}, ContentPadding → True, DefaultBaseStyle → {},
```

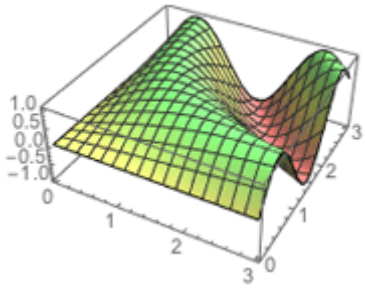
Automatic 是常见的默认设置，表示“自动选择”.

某些选项经常以纯函数给出. 当使用 & 时，需要圆括号：

In[3]:=

```
Plot3D[Sin[x y], {x, 0, 3}, {y, 0, 3},
  ColorFunction → (Hue[#3 / 3, .5] &)]
```

Out[3]=




---

使用 *option:>value* 使得每次使用选项时，*value* 会被重新计算.

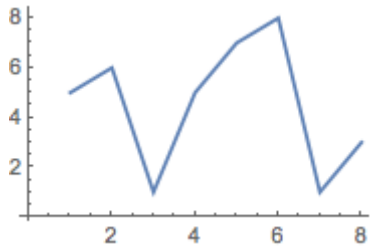
## 2.14 图形

Wolfram 语言可以很容易地提取任何类型的数据并对数据可视化：

In[1]:=

```
ListLinePlot[{5, 6, 1, 5, 7, 8, 1, 3}]
```

Out[1]=




In[2]:=

```
Graph[{1 → 3, 1 → 2, 2 → 4, 4 → 5, 5 → 1}]
```

Out[2]=

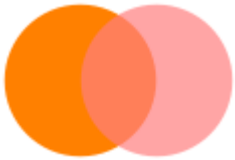
> *PYTHON* 程序员备注：
> Wolfram 语言对图像进行了深度整合，既可以直接在 Wolfram 笔记本中显示也可以作为输入使用. Python 仅支持文本输入，并需要第三方笔记本库，例如 **Jupyter** 或图像库，例如 **matplotlib** 来显示图像.

图形使用“指令”或“样式”，并以符号表达式表示：

In[3]:=

```
Graphics[{Orange, Disk[{0, 0}], Opacity[.7], Pink, Disk[{1, 0}]}]
```

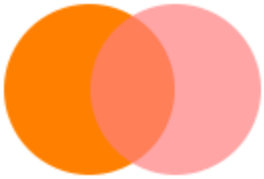
Out[3]=



In[4]:=

```
Graphics[{
  Style[Disk[{0, 0}], Orange], Opacity[.7], Pink, Disk[{1, 0}]}]
```

Out[4]=



快速参考：图形和可视化

---

可视化函数有多种选项.
PlotTheme 提供了一种选择全局“主题”的方法.

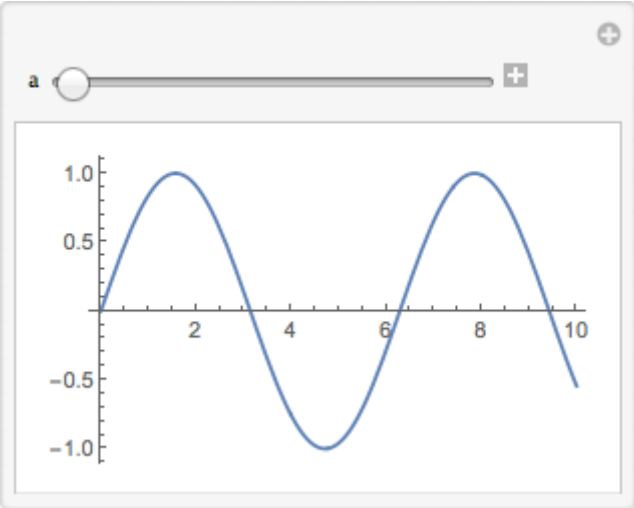
## 2.15 交互式界面

您可以使用任何表达式，并通过控件改变变量值进行“操纵”：

In[1]:=

```
Manipulate[Plot[Sin[a x], {x, 0, 10}], {a, 1, 5}]
```

Out[1]=



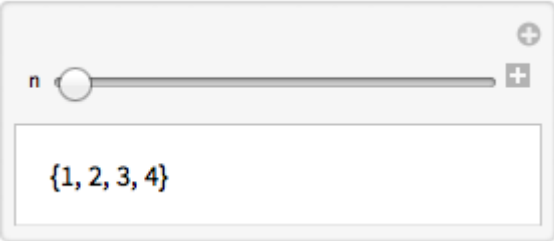

> *PYTHON* 程序员备注：
> **Manipulate** 在 Wolfram 语言中极大简化了 GUI 的生成，根据参数规格自动推断控制类型和排版，并且相同代码在桌面、云端和移动设备间可无缝运行. 在 Python 中，构建交互式界面需要使用用于独立 GUI 的代码或第三方笔记本库. 在 Python 中创建用于桌面和云端的相同界面需要使用适用于各平台的不同代码.

Manipulate 使用标准的 *min*、*max*、*step* 迭代器规范：

In[2]:=

```
Manipulate[Range[n], {n, 4, 10, 1}]
```

Out[2]=



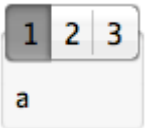

---

在 Wolfram 语言中，控件只是个符号表达式：

In[1]:=

```
TabView[{a, b, c}]
```

Out[1]=

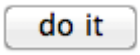



通过给定一个表达式制作一个按钮，当按下按钮时进行计算：

In[2]:=

```
Button["do it", Speak["hello"]]
```

Out[2]=




快速参考：交互式控件

---

Dynamic 显示为表达式的动态更新值.
（这对于本地桌面版本总是可能的；但是在云中的支持可能是有限的.）

移动滑动条改变 *x* 的值：

In[1]:=

```
Slider[Dynamic[x]]
```

Out[1]=

显示当前值：

In[2]:=

```
x
```

Out[2]=

0.422

总显示当前值:

In[3]:=

```
Dynamic[x]
```

Out[3]=

0.422

> *PYTHON 程序员备注:*
> **Dynamic** 一个强大且独立的 Wolfram 语言构建，在任何改变发生时，其会动态更新显示输出，而且它还构成了 **Manipulate** 的基本. 在 Python 中, 没有直接类似的功能，但你可以通过 **ExternalEvaluate** 将 Python 代码整合于 Wolfram 语言.

## 2.16 过程

> *PYTHON 程序员备注:*
> Wolfram 语言规整的整合了过程式、对象导向和函数式编程，让你可以自由选择并结合策略用于不同项目中.

在 Wolfram 语言中通常只需要少量的过程式编程.
（若你的大型程序中如果含有大量 If、Do、Return 等，则该程序可能需要改进.）

使用 ; 分隔不同的操作:

In[1]:=

```
Print[a]; Print[b]; Print[c]
```

Out[1]=

a
b
c

> *PYTHON 程序员备注:*
> 您可以用 Wolfram 语言用任意方式在行间安排过程. 而 Python 则不能强制特定空白和代码样式规则.

---

Module 使用词法作用域（局部化名称）.

Block 使用动态作用域（局部化值）.

DynamicModule 使用文档中的作用域.

Sow/Reap 和 Throw/Catch 是过程式编程中传递数据和控制的有用方式.

> *PYTHON 程序员备注:*
> **Throw** 和 **Catch** 函数在 Wolfram 语言中与 Python 的 **try/except** 结构类似，但更加广泛. 对于特定错误处理，Wolfram 语言还包含了 **Check** 函数.

---

True 和 False 是 Wolfram 语言中的符号.

In[1]:=

```
7 > 5
```

Out[1]=

```
True
```

> *PYTHON 程序员备注:*
> Wolfram 语言使用常见符号 **True** 和 **False**; 在 Python 中, 对于每个对象必须分别定义一个 ***bool*** 属性.

---

在 Wolfram 语言中，If 仅为另一个内置函数:

In[1]:=

```
If[7 > 5 && 2 ≤ 4, a, b]
```

Out[1]=

```
a
```

> *PYTHON 程序员备注:*
> **While** 和 **If** 在 Wolfram 语言中与 Python 中的 **while** 和 **if** 类似，但可以处理符号值.

## 2.17 数值

> *PYTHON 程序员备注:*
> Python 用 Wolfram 客户端库自身支持多种 Python 数字类型，其中包括 NumPy 的 **integer** 和 **float16** 等.

默认情况下，Wolfram 语言会尽可能地进行精确计算:

In[1]:=

```
3/7+2/11
```

Out[1]=

$$\frac{47}{77}$$

> *PYTHON 程序员备注:*
> 在 Wolfram 语言中有理数作为既约分数以符号形式表示. Python 中的类似功能需要导入 **fractions** 模块.

使用 N（可能更快）获取数值结果:

In[2]:=

```
N[3/7+2/11]
```

Out[2]=

```
0.61039
```

---

Wolfram 语言可以处理任何精度的数值:

In[1]:=

```
N[Pi, 50]
```

Out[1]=

3.14159265358979323846264338327950288419716939937 51

可自动跟踪结果精度.

用 ` 以数值明确指明精度:

In[2]:=

```
1.234`50
```

Out[2]=

1.23400000000000000000000000000000000000000000000000

> *PYTHON* 程序员备注:
> Wolfram 语言中的数字可以像 Python 整数可变为任意大小. Floats 在 Python 中需要导入第三方库 **mpmath**.

---

对于复数, I 表示 $\sqrt{-1}$ :

In[1]:=

```
I^2
```

Out[1]=

$-1$

> *PYTHON* 程序员备注:
> Wolfram 语言提供了数量不同的格式化形式, 例如 $i$、$j$ 和 $\sqrt{-1}$ 用于虚数, 并且与其他常数类似, 来最大化明确性. 另外, 无论数字是否为实数或复数, Wolfram 语言会在执行时候自动进行推论. Python 默认使用符号 **j** 来表示一个虚数, 且在通用运算符中并不会一定会被运用.

---

矩阵是列表的列表:

In[1]:=

```
Inverse[{{6, 7}, {4, a}}]
```

Out[1]=

$$\left\{\left\{\frac{a}{-28+6a}, -\frac{7}{-28+6a}\right\}, \left\{-\frac{4}{-28+6a}, \frac{6}{-28+6a}\right\}\right\}$$

---

SparseArray 给出稀疏数组.

> *PYTHON* 程序员备注:
> 在 Wolfram 语言中, 用例如 **MatrixForm** 的函数可以简单的在数组和矩阵中进行移动. 在 Python 中, 默认并未提供稀疏数组并且需要第三方程序库. 此外, 由于是不同的等级, 这将会造成在 **SciPy** 中的矩阵和 **NumPy** 中的数组间移动时引起问题.

## 2.18 字符串

Wolfram 语言中的字符串用双引号指定 ("...") .

当输出为字符串时, 没有双引号, 除非使用 InputForm:

In[1]:=

```
"this is a string"
```

Out[1]=

```
this is a string
```

In[2]:=

```
InputForm[%] ← % 指的是前一个输出
```

Out[2]=

```
"this is a string"
```

用 <> 连接字符串：

In[3]:=

```
"this " <> "is " <> "aaaa" <> "bbb"
```

Out[3]=

```
this is aaaabbb
```

> *PYTHON* 程序员备注：
> <> 在 Wolfram 语言中特别用于连接字符串，目的是为避免使用 + 而产生模糊.

---

字符串可以包含任何 Unicode 字符，输入名字可以是 \[Alpha]，快捷键 ESC A ESC，明确的 Unicode，例如 \:03b1，或从面板中输入 α.

> *PYTHON* 程序员备注:
> Python 3 和 Wolfram 语言字符串都默认使用 Unicode. 在 Python 2 中，除非特别定义为 Unicode，字符串默认使用 ASCII.

---

字符串模式使用 ~~ 组合带有模式结构的字符串：

In[1]:=

```
StringReplace["this is a string",
  Shortest[LetterCharacter .. ~~ "is"] → "xxx"]
```

Out[1]=

```
xxx xxx a string
```

模式 *p*... 中指明 *p* 的零次或更多次重复.
RegularExpression 允许紧凑的正则表达式符号.

> *PYTHON* 程序员备注：
> Wolfram 语言字符串模式用普通模式语法提供了强大的字符串匹配, 对于可读性和直观性进行了优化. Python 中最接近的是用 regex 的字符串匹配，其与 Wolfram 语言的 **RegularExpression** 函数作用相近.

---

字符串模版 使用 `` 指明"插入符"，并用 <* ... *> 指明要计算的表达式：

In[1]:=

```
TemplateApply["number `` date: <* Now *>", {56}]
```

Out[1]=

```
number 56 date: Sun 1 Jun 2014 12:30:44
```

> *PYTHON 程序员备注：*
> **TemplateApply** 在 Wolfram 语言中与 Python 的 **%** 或 **format** 函数的作用类似，除了 Python 中不支持用 **<* ... *>** 进行行内计算.

## 2.19 关联

> *PYTHON 程序员备注：*
> **Associations** 在 Wolfram 语言中与 Python 的字典作用类似. 使用 **Association**，程序员可以用高效的查找和更新将键值相关联，即使有上百万种类元素，同时还需要在不使用例如 Python 的 **OrderedDict** 的特定的数据结构来保持这些元素的插入顺序. 当通过 Python 用 Wolfram 客户端库进行 Python 编程时，**dict**、**OrderedDict** 和 **Series**（来自 **Pandas** 库）等类似分类对 **Association** 串行化.

关联将把键符与其值相关联：
（用 `->` 输入 →）

In[1]:=

```
<|"a" → x, "b" → y|>
```

Out[1]=

```
<|a → x, b → y|>   ← 字符串引号在输出中没有显示
```

（关联中的键经常使用字符串，但它不是必需的.）

将关联应用于一个键给出对应的值：

In[2]:=

```
%["a"] ← % 指的是前一个输出
```

Out[2]=

```
x
```

---

在纯函数中，`# key` 选出在关联中对应于"*key*"的值：

In[1]:=

```
{#b, 1 + #b} & [<|"a" → x, "b" → y|>]
```

Out[1]=

```
{y, 1 + y}
```

---

你可以混合关联和列表，用 `[[ ... ]]` 选出部分：

In[1]:=

```
<|"a" → x, "b" → {5, 6}|>[["b", 1]]
```

Out[1]=

字符串模板适用于关联（与 XML 和笔记本模板相同）：

In[1]:=

```
TemplateApply["first `a`; second `b`; first `a`", <|"a" → x, "b" → y|>]
```

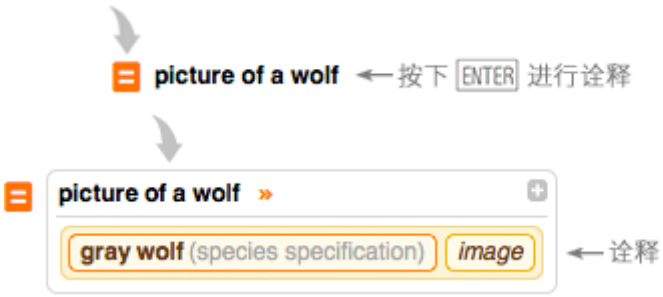
Out[1]=

```
first x; second y; first x
```

## 2.20 自然语言输入

在行首敲入 = 指定自然语言输入：

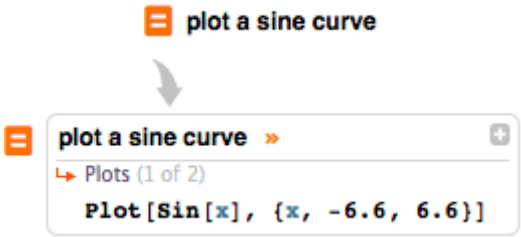
In[1]:=



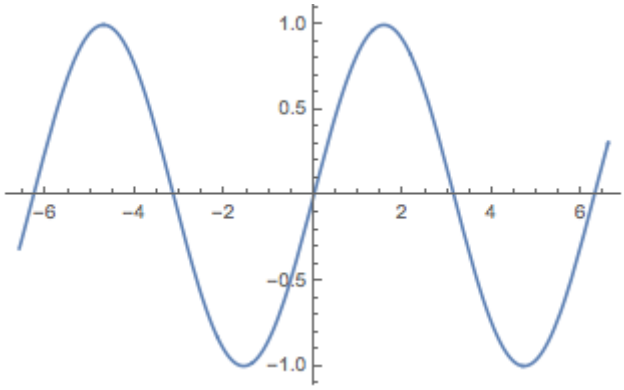

Out[1]=

很多时候，你可以从自然语言中产生代码：

In[2]:=




Out[2]=




---

用 `==` 获取完整的 Wolfram|Alpha 报告:

In[1]:=

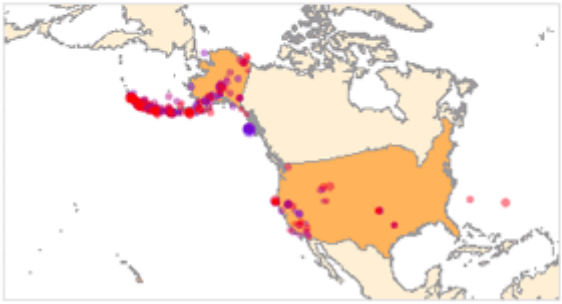
Out[1]=

Input interpretation:

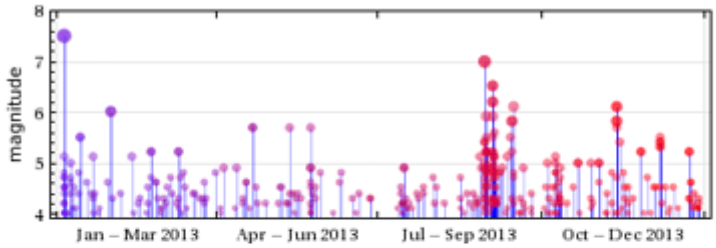
| | |
|---|---|
| earthquakes | United States |
| | last year |

Results:

Magnitude > 4 | CDT | Show local map



Timeline:




List:

Sort by magnitude | More | Show coordinates | Show metric

| magnitude | time | location |
|---|---|---|
| 7.5 | Sat, Jan 5, 2013 02:58 am CST (1.4 years ago) | 56 mi WSW of Edna Bay, Alaska, United States |
| 7.0 | Fri, Aug 30, 2013 11:25 am CDT | 58 mi ESE of Adak, Alaska, United States |

# 2.21 真实世界实体

在 Wolfram 语言中，真实世界实体只是另一种符号表达式.

Wolfram 语言知道数千种真实世界实体:

国家 城市 化学品 物种 电影 人物 卫星 机场 公司 ...

使用自然语言指定实体很方便:

In[1]:=

Los Angeles (city)

Out[1]=

Los Angeles

实体有很多属性. 这是其中的一个值:

In[2]:=

EntityValue[Los Angeles (city), "Population"]

Out[2]=

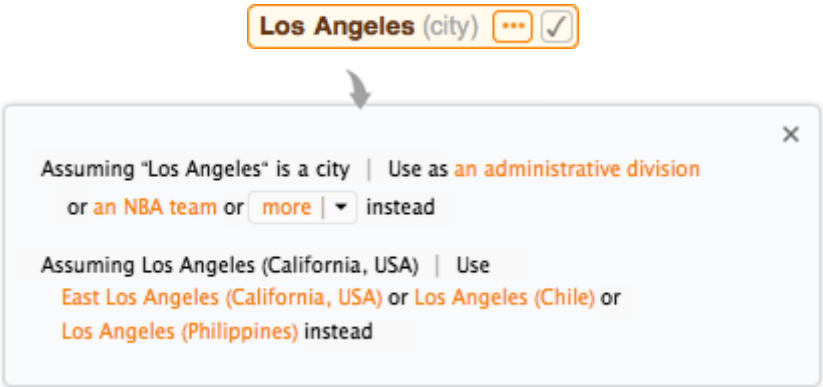
3 857 799 people

用 *entity*["Properties"] 找到属性列表.

---

当输入自然语言时，⋯ 用于消除歧义:

In[1]:=




---

⁞⁞ 表示一个实体类:

In[1]:=

EntityList[⁞⁞ German speaking countries (countries)]

Out[1]=

{ Austria , Namibia , Luxembourg , Belgium ,
Germany , Italy , Liechtenstein , Switzerland }

---

使用 CTRL + = 输入单位和度量：

3 meters

In[1]:=

3 m

Out[1]=

3 m

InputForm 显示符号表达式的结构：

In[2]:=

InputForm[%] ← % 指的是前一个输出

Out[2]=

Quantity[3, "Meters"]

---

GeoPosition 代表一个地理位置：

In[1]:=

GeoPosition[ Eiffel Tower (building) ]

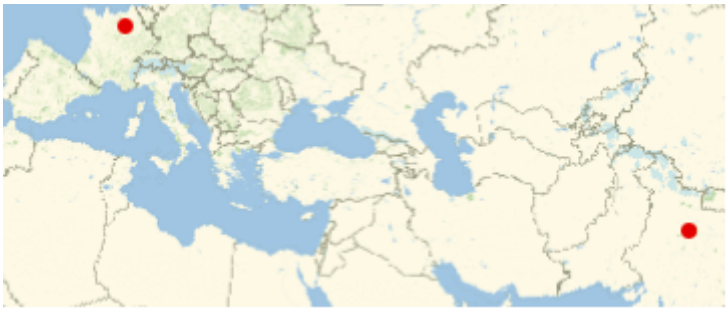
Out[1]=

GeoPosition[{48.8583, 2.29444}]

In[2]:=

GeoListPlot[ { Eiffel Tower (building) , Uluru (mountain) } ]

Out[2]=



---

DateObject 代表日期/时间：

In[1]:=

```
Sunset[Tomorrow]
```

Out[1]=

Mon 2 Jun 2014 21:18 GMT−4.

In[2]:=

```
InputForm[%]
```

← % 指的是前一个输出

Out[2]=

```
DateObject[{2014, 6, 2}, TimeObject[{21, 18}], TimeZone -> -4.]
```

## 2.22 解释器

用 Interpreter 解释自然语言的很多类型和其他输入：

数字 日期 颜色 位置 语言 电影 化学品 图像 ...

In[1]:=

```
Interpreter["Date"]["jun 23, 1988"]
```

Out[1]=

Thu 23 Jun 1988

In[2]:=

```
Interpreter["City"]["LA"]
```

Out[2]=

Los Angeles

---

Interpreter 处理需要诠释意义的“语义”类型：

In[1]:=

```
Interpreter["SemanticNumber"]["three quarters"]
```

Out[1]=

$$\frac{3}{4}$$

它也处理需要计算的类型：

In[2]:=

```
Interpreter["ComputedCity"]["capital of poland"]
```

Out[2]=

Warsaw

---

Restricted 让你限制允许的输入（例：仅在某个范围的数值）.

## 2.23 云部署

CloudDeploy 将对象部署于 Wolfram Cloud.

创建一个 “hello” 字体大小为 100 点的网页：

In[1]:=

```
CloudDeploy[Style["hello", 100]]
```

Out[1]=

```
CloudObject[https://www.wolframcloud.com/objects/69f7275b-86c4 ...]
```

云对象以一个可以浏览的唯一网址形式给出：

---

部署一个动态界面：

In[1]:=

```
CloudDeploy[
  Manipulate[Rotate[Style["hello", 100], θ °], {θ, 0, 360}]]
```

Out[1]=

```
CloudObject[https://www.wolframcloud.com/objects/f54f66e4-b9f2...]
```

该界面可用于任何浏览器：

---

部署 笔记本的任何内容（动态或非动态），并保留其样式.

使用 EmbedCode 设置 iframes 等的内嵌.

使用 CloudDeploy[Delayed[...]] 部署每次请求时会被重新计算的表达式.

---

创建和部署表单界面:

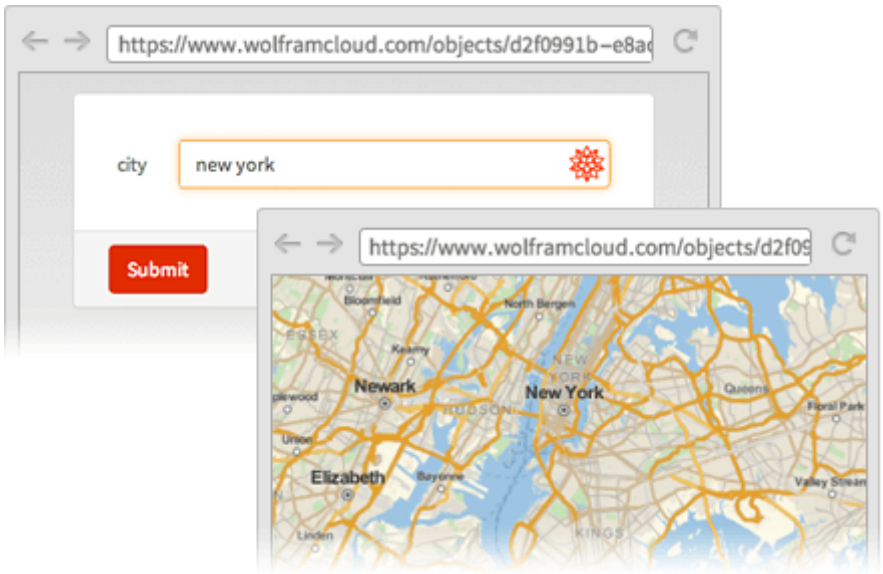
字段名 类型 函数 格式

In[1]:=

```
CloudDeploy[FormFunction[{"city" → "City"}, GeoGraphics[#city] &, "PNG"]]
```

Out[1]=

```
CloudObject[https://www.wolframcloud.com/objects/d2f0991b-e8ac...]
```



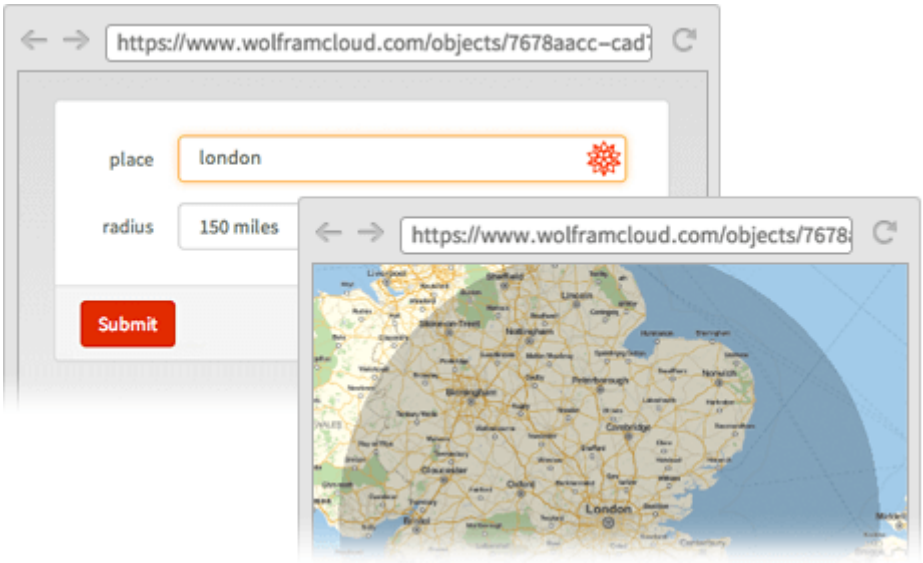

任何 Interpreter 类型可被用于表单中的一个字段:

In[2]:=

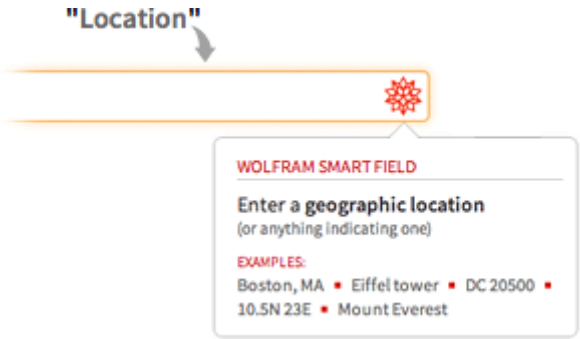
```
CloudDeploy[
  FormFunction[{"place" → "Location", "radius" → "Quantity"},
    GeoGraphics[GeoDisk[#place, #radius]] &, "PNG"]]
```

Out[2]=

```
CloudObject[https://www.wolframcloud.com/objects/7678aacc-cad7...]
```




语义类型会产生诠释自然语言的 “智能搜索栏”：




在云端部署一个网页 API：

参数 类型 函数 格式

In[1]:=

```
CloudDeploy[APIFunction[{"x" → "Number"}, #x! &, "String"]]
```

Out[1]=

```
CloudObject[https://www.wolframcloud.com/objects/79c6609f-a600 ...]
```

调用 API：

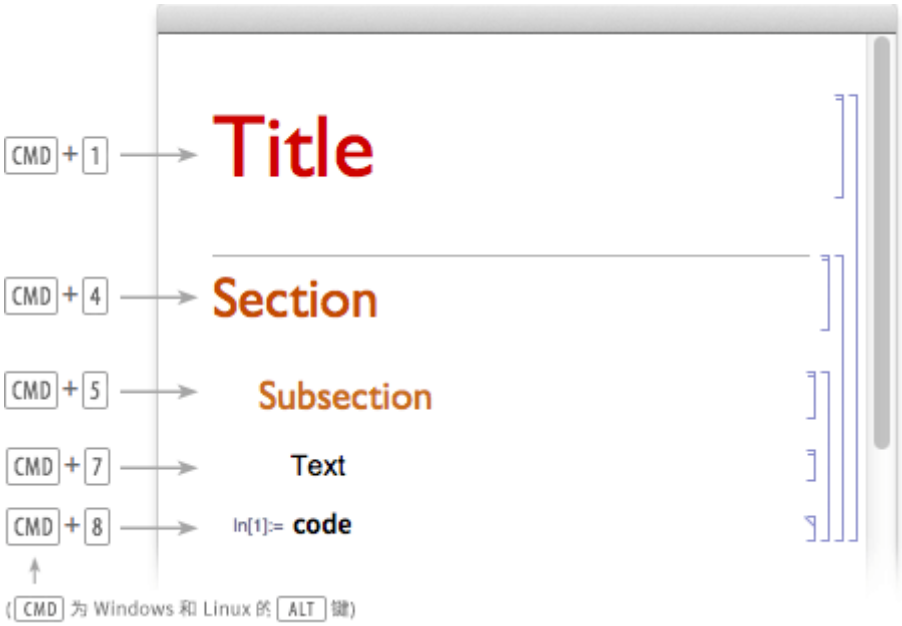
参数 值

https://www.wolframcloud.com/objects/79c6609f-a600...2e35f708?x=1234


```
5108498146646957688130617626100459875027274162463620787575836488567978388638911 4
1199043673982149094516168659597971900855959572160602010817908635627407113924084 0
2606162284424347926444168293770306459877429620549980121621880068812119922825565 6
0375003679365742847649857731688789068928488446442352246916292465441994549694005 2
7460669508677840847535815401481943168883038396948608703570082355250281152814023 7
9270279446743097868896180567901452872031734195056432576568754346528258569883526 8
5982672773583865408224672175181965805269239627061134801301378673932022970600994 0
7810255860388094930139921110304324733215322285896361507226213603669786074846928 7
0955691740723349227220367512994355146567475980006373400215826077949494335370591 6
2367114202695792393766922477161716795935965043996639267307318013937656307370656 2
```

## 2.24 制作文档

在笔记本中制作标题、章节和文本单元等：




> *PYTHON* 程序员备注：
> Wolfram 语言笔记本本身支持完整的格式和样式，用于创建在桌面、移动设备和 Wolfram Cloud 网页中的报告、演示和其他丰富、计算性交互文档. 在三十多年的开发下，Wolfram 语言笔记本为其他笔记本界面提供了灵感，例如静态的网页 **Jupyter**，虽然其缺少分级结构和折叠单元的功能.

---

用 Grid 布局一个网格：

In[1]:=

```
Grid[{{a, b}, {c, d}}, Frame → All]
```

Out[1]=

| | |
|---|---|
| a | b |
| c | d |

---

当可用时，**新建** ▶ **模板笔记本** 让你创建一个模板笔记本：

GenerateDocument 从模板中产生一个文档，计算模板表达式并从提供的关联中填充模板插符.

> *PYTHON 程序员备注：*
> 模板笔记本与 Wolfram 语言中的其他结构同样可用符号操作. 附加新信息可用多种内置函数规整有效地填充模板插符，例如 **TemplateApply**，可用于创建文档.

## 2.25 管理会话

% 代表最近的输出; %*n* 代表结果 Out[*n*].

> *PYTHON 程序员备注：*
> Wolfram 语言中的 **%** 与 IPython 所用的 **%** 或 **%%** 相同，但是必须在 Python 上另外进行安装 IPython. 另外，你可以即刻用其进行计算并操作其他部分代码中的 **%n**. 这在 IPython 中是不行的.

使用 `CMD`+`L` （`ALT`+`L`）将最近的输入复制到当前输入.

---

若在行尾使用 `;` 则不会显示输出：

In[1]:=

```
t = Sunset[];
```

> *PYTHON 程序员备注：*
> 除了尾行之外，Wolfram 语言中的分号可以限制显示输出.

% 仍然适用：

In[2]:=

```
% ← % 指的是前一个输出
```

Out[2]=

```
Thu 19 Jun 2014 20:26 GMT−5.
```

---

使用 Short 获取输出的简短摘要：

In[1]:=

```
Short[Range[1000]]
```

Out[1]=

```
{1, 2, 3, 4, 5, 6, 7, ≪986≫, 994, 995, 996, 997, 998, 999, 1000}
```

使用 *//function* 将一个函数应用于尾行：

In[2]:=

```
Range[1000] // Short
```

Out[2]=

```
{1, 2, 3, 4, 5, 6, 7, ≪986≫, 994, 995, 996, 997, 998, 999, 1000}
```

---

如果你需要结束计算，在菜单选项中选择放弃计算，或按 CMD+.（Mac），或 ALT+.（其他）.

# 2.26 外部连接

Import 与 Export 自动处理来自文件、程序、网页等的数百种格式：

导入元素

In[1]:=

```
Import["https://www.nps.gov", "Images"]
```

Out[1]=

> *PYTHON* 程序员备注：
> Wolfram 语言用 **Import** 和 **Export** 函数统一了上百种不同的导入和导出的文件类型. Python 的文件格式支持则区别于不同程序包和程序库，通常含有不同语义的数据表述.

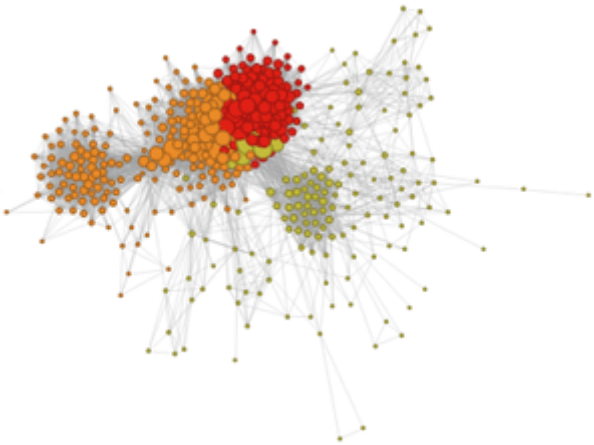
---

Wolfram 语言精选许多外部 API 连接：

In[1]:=

```
SocialMediaData["Facebook", "FriendNetwork"]
```

Out[1]=



---

The 设备框架 允许来自于 连接设备的直接输入：

In[1]:=

```
DeviceReadTimeSeries["GPIO", {10, 1}, 4]
```

Out[1]=

TimeSeries[ Time: 17:37:56.945 to 17:38:05.959 Data points: 10 ]

> *PYTHON 程序员备注：*
> Wolfram 语言的内置监管里连接和配置允许无缝访问上千种设备和外部 APIs，在 Python 中则需要更多工具和程序库来实现.

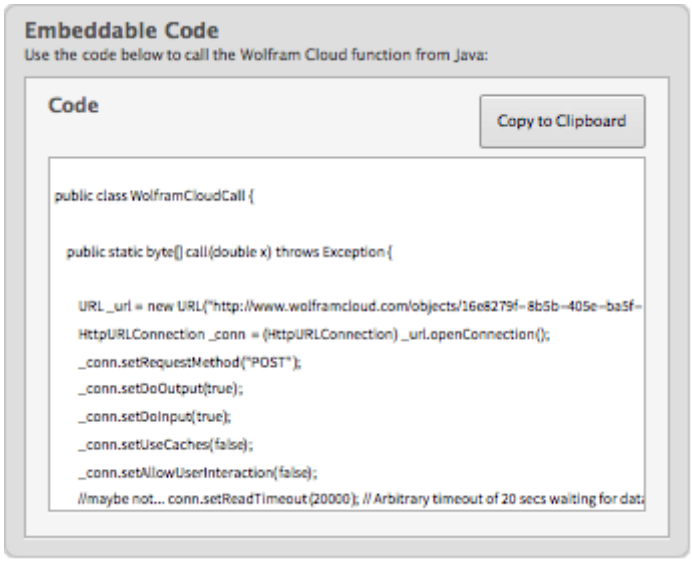
---

使用 EmbedCode 创建从其他语言调用一个 即时 API 的代码：

In[1]:=

```
EmbedCode[APIFunction[{"x" → "Number"}, #x! &, "PNG"], "Java"]
```

Out[1]=




---

对于介于 Wolfram 引擎和其他程序间的双向通讯使用 WSTP.
使用 C/Link、J/Link、等等. 创建外部代码与 WSTP 进行交互.

> *PYTHON 程序员备注：*
> 除使用 WSTP 和调用即时 API 之外，Wolfram 语言还可通过 Python 用 Wolfram 客户端库或使用 **WolframScript** 从 Python 中调用来计算本地或基于云端的 Wolfram 引擎.

---

在行的起始键入 `>` 指定外部计算的系统（Python、NodeJS等）：

计算器解释输入并返回 Wolfram 语言表达式：

In[1]:=

```
[i/2 for i in range(10)]
```

Out[1]=

```
{0, 0, 1, 1, 2, 2, 3, 3, 4, 4}
```

用 ExternalEvaluate 来指定计算器的选项：

In[2]:=

```
ExternalEvaluate[{"Python", "Version" → "3.6"},
 "[i/2 for i in range(10)]"]
```

Out[2]=

```
{0., 0.5, 1., 1.5, 2., 2.5, 3., 3.5, 4., 4.5}
```

> *PYTHON* 程序员备注：
> **ExternalEvaluate** 支持 Python 2.4+ 和 3.6+，以及对 **NumPy** 和 **PIL** 安装包的的直接整合.